

Final draft Spec_for FogP...
PDF

go through this and What is this project actually doing?
Explain the complete flow step by step? How do I build
this from scratch? Give project folder structure
Create roadmap for development and What should I learn
before starting this project?

What is this project actually doing?

This project is a railway safety assistance system called **FogPASS (Fog Pilot Assistance System for Safety)**. It is designed for Indian Railway loco pilots (train drivers) to help them during foggy weather or low visibility conditions.

Final draft Spec_for FogPASS Ve...

The system is basically:

- A rugged tablet device installed inside a train cabin
- Connected with GPS/GNSS
- Loaded with railway route data and landmarks
- Gives:
 - voice alerts
 - visual warnings
 - distance information
 - speed information

before the train reaches important railway landmarks like:

- signals
- level crossing gates
- speed restrictions
- neutral zones
- bridges
- curves
- halt stations

The purpose is to prevent mistakes during low visibility and improve train safety.

Final draft Spec_for FogPASS Ve...

Real-World Example

Imagine:

- Heavy fog
- Driver cannot clearly see upcoming signal
- Train moving at 100 km/h

FogPASS says:

“Gorakhpur Home Signal approaching — 500 meters”

and shows:

- signal name
- distance remaining
- next two landmarks
- train speed
- route direction

This helps the loco pilot stay alert.

Core Idea of the System

The system continuously:

1. Reads GPS coordinates from train location
2. Matches current position against stored railway route map
3. Calculates nearest landmarks
4. Checks warning distance
5. Announces alerts
6. Displays remaining distance
7. Logs all events

Complete System Flow (Step-by-Step)

Phase 1 — Route Preparation

Before train operation:

Railway team performs route survey.

They collect:

- signal coordinates
- LC gate coordinates
- PSR coordinates
- station coordinates
- gradients
- route directions

Stored in CSV route files. Final draft Spec_for FogPASS Ve...

Example:

Landmark	Latitude	Longitude	Type
Signal A	26.xxxx	80.xxxx	Home Signal
LC Gate	26.xxxx	80.xxxx	LC

Phase 2 — Load Route Data into Device

Data gets loaded into FogPASS using:

- USB
- Laptop/PC
- WiFi
- Pen drive

Final draft Spec_for FogPASS Ve...

The device stores:

- multiple routes
- landmark data
- voice files
- warning configurations

Phase 3 — Device Startup

When driver powers ON:

System:

1. boots OS
2. initializes GPS
3. loads route database
4. shows software version
5. displays safety disclaimer

Final draft Spec_for FogPASS Ve...

Then driver:

- enters train number
- enters user ID
- selects foggy/non-foggy mode

Phase 4 — GPS Position Tracking

Device continuously receives:

- latitude
- longitude
- speed
- direction

from GNSS receiver.

Final draft Spec_for FogPASS Ve...

System then:

- identifies current track
- detects route
- determines travel direction
- finds nearest landmarks

Phase 5 — Landmark Matching Engine

This is the heart of the project.

The engine continuously:

```
Current Train Position
↓
Compare with Route Database
↓
Find upcoming landmarks
↓
Calculate distance
↓
Trigger warning if needed
```

Example:

```
Train Position:
26.4500, 80.2200
```

```
Upcoming Landmark:
Home Signal
```

```
Distance:
480m
```

If within warning distance:

- alert generated

Phase 6 — Voice Warning System

When train enters warning zone:

System announces:

```
"Gorakhpur Home Signal"
```

or

```
"LC Gate approaching"
```

Final draft Spec_for FogPASS Ve...

Voice files stored locally:

- mp3
- wav

Phase 7 — Display System

Tablet UI displays:

Main Landmark

- name
- type
- remaining distance

Next 2 Landmarks

- smaller font
- queued sequence

Final draft Spec_for FogPASS Ve...

Also shows:

- GPS speed
- battery
- GPS status
- route direction
- fog mode

Final draft Spec_for FogPASS Ve...

Phase 8 — Event Logging

System stores logs:

- landmarks crossed
- speed
- warnings
- snooze actions
- timestamps
- GPS failures

Final draft Spec_for FogPASS Ve...

Used later for diagnostics and reporting.

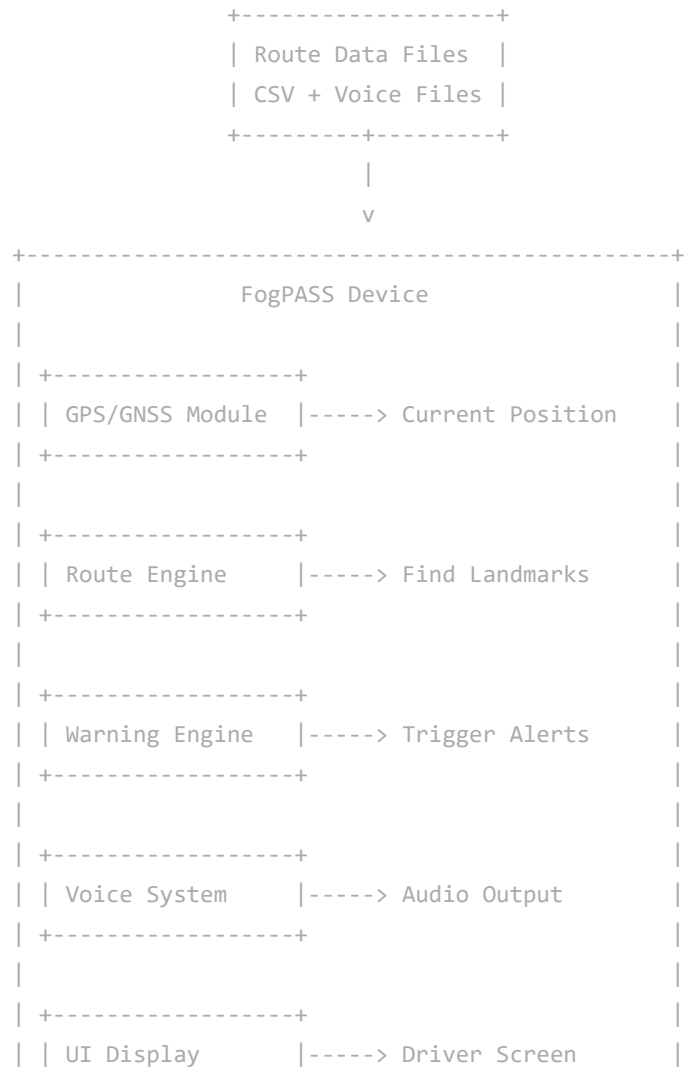
Phase 9 — Diagnostics

Self-diagnostic system checks:

- GPS module
- antenna
- memory
- sensors
- battery

Final draft Spec_for FogPASS Ve...

High-Level Architecture





Technologies You Need

1. Java Fundamentals

Learn:

- OOP
- collections
- threads
- file handling
- networking
- exceptions

Very important because:
this is a real-time system.

2. Java Swing / JavaFX

Needed for:

- tablet UI
- dashboard
- warnings
- display rendering

You already know Swing basics, which helps.

3. GPS/GNSS Concepts

Learn:

- latitude/longitude
- geo-distance calculation

- GPS parsing
- NMEA protocol

Spec mentions NMEA-0183.

Final draft Spec_for FogPASS Ve...

4. Multithreading

Critical because system simultaneously:

- tracks GPS
- updates UI
- plays voice
- logs events

5. File Processing

Need to read:

- CSV
- TXT
- MP3/WAV metadata

6. SQLite / Embedded DB

Useful for:

- route storage
- event logs
- configuration

7. Audio Programming

Need:

- audio playback
- voice queue
- warning priority

8. Real-Time System Concepts

Learn:

- polling
- buffering
- timing
- event-driven architecture

9. Linux Basics

Because actual embedded device may run Linux/Android.

Recommended Tech Stack

Module	Technology
UI	Java Swing / JavaFX
Core Engine	Java
GPS Parser	Java Serial/NMEA
Database	SQLite
Audio	Java Sound API
Route Files	CSV
Logs	SQLite/CSV
Packaging	Java JAR
OS	Linux/Android

Suggested Folder Structure

```
FogPASS/
|
|─ app/
|   |─ Main.java
|   |─ Launcher.java
|
|─ core/
|   |─ gps/
|   |   |─ GPSReader.java
|   |   |─ NMEAParser.java
|   |   └─ Coordinate.java
|   |
|   |─ route/
|   |   |─ RouteManager.java
|   |   |─ Landmark.java
|   |   |─ Route.java
|   |   └─ RouteLoader.java
|   |
|   |─ warning/
|   |   |─ WarningEngine.java
|   |   |─ DistanceCalculator.java
|   |   └─ AlertManager.java
|   |
|   |─ audio/
|   |   |─ AudioPlayer.java
|   |   |─ VoiceQueue.java
|   |   └─ VoiceManager.java
|   |
|   |─ logging/
|   |   |─ EventLogger.java
|   |   └─ LogManager.java
|   |
|   |─ diagnostics/
|   |   |─ SelfTest.java
|   |   └─ DeviceHealthMonitor.java
|   |
|   └─ config/
|       |─ AppConfig.java
|       └─ SettingsManager.java
|
|─ ui/
|   |─ screens/
|   |   |─ HomeScreen.java
|   |   |─ RouteSelectionScreen.java
|   |   |─ WarningScreen.java
|   |   └─ DiagnosticsScreen.java
|   |
|   └─ components/
```

```
| | | └─ SpeedPanel.java
| | | └─ LandmarkPanel.java
| | | └─ StatusBar.java
| | |
| | └─ styles/
| |
| └─ database/
|   └─ SQLiteManager.java
|   └─ schema.sql
|
| └─ resources/
|   └─ routes/
|     └─ sample_route.csv
|   |
|   └─ voices/
|     └─ GKP.mp3
|     └─ NDLS.mp3
|   |
|   └─ icons/
|   |
| └─ simulation/
|   └─ RouteSimulator.java
|   └─ FakeGPSGenerator.java
|
| └─ tests/
|
| └─ docs/
```

Development Roadmap

Phase 1 — Learn Basics

Learn:

- Java OOP
- Swing/JavaFX
- Threads
- File handling
- GPS basics

Duration:

2–4 weeks

Phase 2 — Build Route System

Build:

- CSV route loader
- landmark model
- route manager

Goal:

```
Load route and print upcoming landmarks
```

Phase 3 — GPS Simulation

Create fake GPS generator.

Simulate:

```
Train moving from A → B
```

Goal:

- test without real GPS hardware

Phase 4 — Distance Engine

Implement:

- Haversine formula

to calculate:

```
distance(current, landmark)
```

Phase 5 — Warning Engine

Build logic:

```
IF distance < warningDistance
```

THEN trigger alert

Phase 6 — UI Development

Build:

- main tablet screen
- landmark display
- speed display
- warning colors

Phase 7 — Voice System

Add:

- MP3 playback
- queue system
- repeat logic
- snooze logic

Phase 8 — Event Logging

Store:

- timestamps
- alerts
- speed
- landmark crossings

Phase 9 — Simulation Mode

Spec requires simulation mode.

Final draft Spec_for FogPASS Ve...

Build:

- fake train movement
- test routes

Phase 10 — Hardware Integration

Finally connect:

- real GPS receiver
- antenna
- rugged tablet

Recommended First MVP

Do NOT build everything initially.

Start with:

MVP Features

- Load CSV route
- Simulate GPS movement
- Calculate distance
- Show upcoming landmark
- Play voice alert

That alone teaches 70% of system architecture.

Biggest Technical Challenges

1. Accurate GPS Matching

Need smart route matching.

2. Real-Time Updates

Everything updates continuously.

3. Multi-threading

UI + GPS + audio + logs together.

4. Railway Route Logic

Track switching and direction detection are complex.

Best Way to Start

Start with this mini version:

```
Laptop App
+
Fake GPS Simulator
+
CSV Route File
+
Voice Alerts
```

After that:

- add UI
- add logging
- add real GPS
- add route selection

Your Learning Order

1. Java OOP
2. Swing UI
3. Threads
4. CSV parsing
5. GPS basics
6. Distance algorithms
7. Audio APIs
8. SQLite
9. Real-time systems
10. Embedded/Linux concepts

What This Project Really Is Technically

This is a combination of:

- GPS navigation system
- railway signaling assistant
- embedded software
- real-time alert system
- mapping engine
- voice guidance system
- rugged industrial tablet application

Very similar in architecture to:

- aviation cockpit alert systems
- vehicle navigation systems
- metro train assistance systems

Most Important Modules

If you want to deeply understand this project:

focus mainly on:

1. GPS tracking engine
2. Landmark matching engine
3. Warning engine
4. Real-time UI updates
5. Voice scheduling logic

Those are the actual core of FogPASS.

       Sources